STAT 427:
Illinois Campus Cluster Usage Analysis

Rushil Bhupta Jiachen Cong Vladislav Fedorov

Rishab Sakalkale Yanlin Wang

Outline

Outline

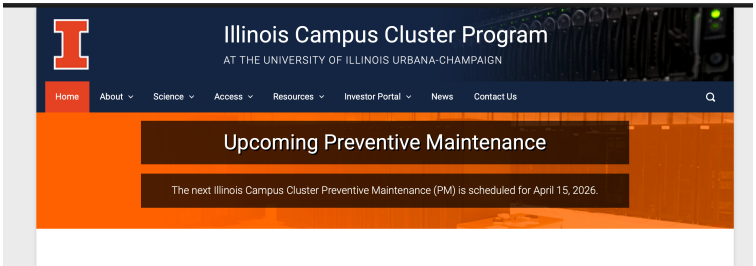
Background and Motivation

The Illinois Campus Cluster

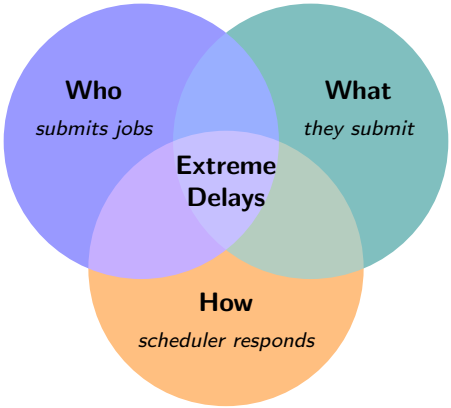
- Campus-wide high-performance computing system
- Users submit jobs to shared compute resources

Motivation

- System performance is highly **heterogeneous**
- Most jobs start quickly, but some experience **extreme delays**
- Resource usage is uneven across workloads and users

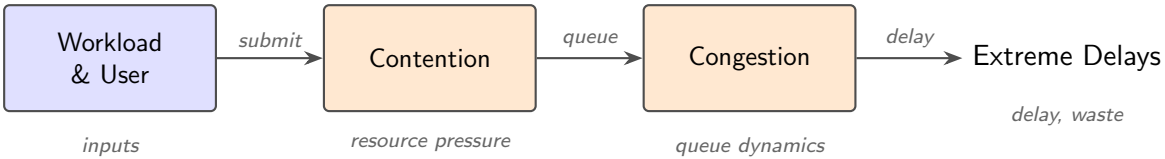


Question: Where Do Extreme Delays Come From?



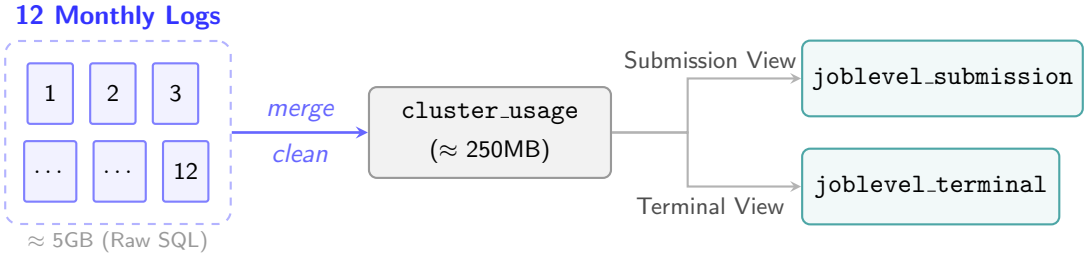
- Extreme delays emerge from the **interaction** of:
  Who submits, *What* is submitted, and *How* the scheduler responds

Analysis Pipeline & Project Objectives



- **Establish system baseline**
  Reveal heavy-tailed behavior in waiting time and runtime distributions
- **Identify sources of heterogeneity**
  Distinguish workload types and user-level concentration
- **Explain extreme delays through the pipeline**
  Show how heavy jobs and timing effects lead to tail congestion
- **Translate insights into action**
  Predict delay-prone jobs and propose scheduling improvements

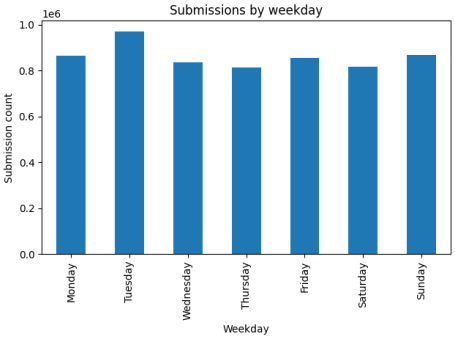
Datasets Construction Pipeline

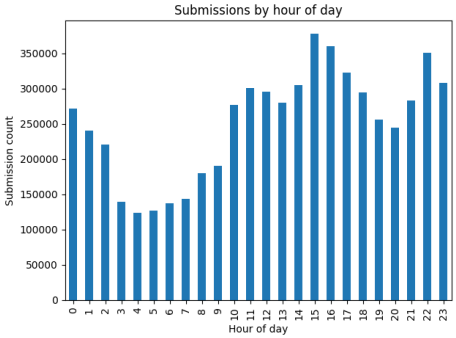


- All downstream analysis mainly focus on these two datasets.

Outline

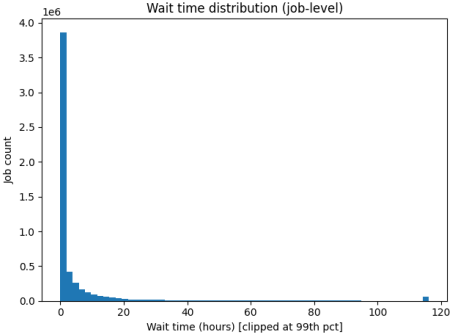
When Are Jobs Submitted?

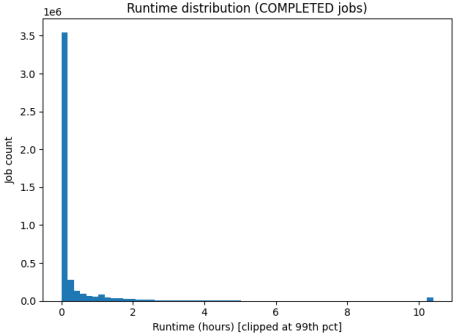




- Submissions are relatively **consistent across weekdays** (slight peak early in the week)
- Hourly pattern: **peaks in afternoon/late evening**, lowest in early morning, slight dip 7–8 P.M.
- Pattern is consistent with student/researcher daily activity.

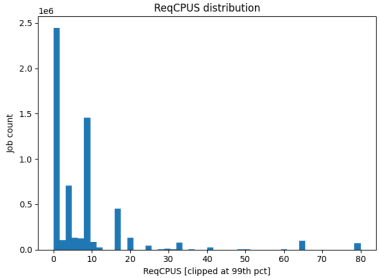
Waittime and Runtime

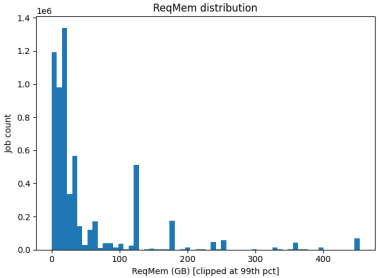


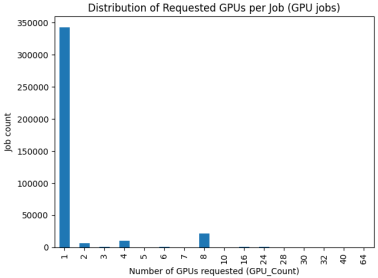


- **Waiting time** highly skewed: median 0.42 hr, but 90th pct $= 13$ hr
- **Runtime** similarly skewed: median 0.02 hr, but 99th pct ≈ 10 hr
- Both distributions have **long tails** that drive the system's behavior

Resource Demand: CPU, Memory, GPU







- Most jobs are **small**: median 4 CPUs, 16 GB memory
- GPU jobs: **89%** request only 1 GPU; only 6% request 5+
- But long tails exist: jobs requesting 40 or 64 GPUs do appear

Job State Distribution

outcome	job_count	job_pct	row_count	row_pct
Cancelled	639041	10.59	641112	10.50
Completed	4644274	76.98	4662342	76.36
Failed	585157	9.70	586554	9.61
Node Failed	218	0.00	220	0.00
Other	845	0.01	850	0.01
Out of Memory	43313	0.72	43546	0.71
Requeued	9	0.00	9	0.00
Running	301	0.00	3600	0.06
Stuck PENDING	3791	0.06	45464	0.74
Timeout	116144	1.93	122413	2.00

Total unique jobs: 6033093
Total rows: 6106110

- 76% of jobs are completed
- 43k jobs are out-of-memory
- 20% of jobs are either cancelled or failed

Takeaway: System failures account for a fraction of incomplete jobs, while job errors (cancellations, failed jobs) are significantly higher. How can we minimize this?

Terminal State Structure

outcome	count	mean	std	min	25%	50%	75%	max
Cancelled	639041.0	1.003241	0.065675	1.0	1.0	1.0	1.0	10.0
Completed	4644274.0	1.003890	0.062251	1.0	1.0	1.0	1.0	2.0
Failed	585157.0	1.002387	0.048943	1.0	1.0	1.0	1.0	3.0
Node Failed	218.0	1.009174	0.095562	1.0	1.0	1.0	1.0	2.0
Other	845.0	1.005917	0.076741	1.0	1.0	1.0	1.0	2.0
Out of Memory	43313.0	1.005379	0.073148	1.0	1.0	1.0	1.0	2.0
Requeued	9.0	1.000000	0.000000	1.0	1.0	1.0	1.0	1.0
Running	301.0	11.960133	0.267841	10.0	12.0	12.0	12.0	12.0
Stuck PENDING	3791.0	11.992614	0.112296	7.0	12.0	12.0	12.0	12.0
Timeout	116144.0	1.053976	0.237671	1.0	1.0	1.0	1.0	3.0

- 1 unique job = 1 row in the SLURM dataset
- Only pending and running states have multiple rows per job

Time-series of cluster load and failures

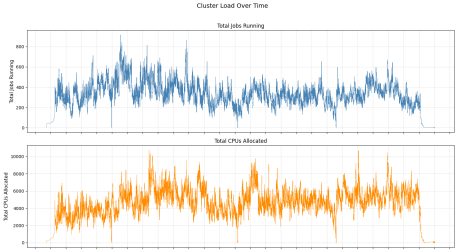


Figure 1: Job State Counts

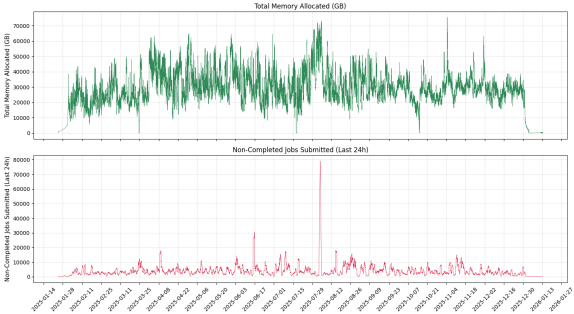


Figure 2: Job State Counts

Section Summary

Key Findings:

1. Job submissions are temporally structured: stable across weekdays, but vary by hour.
2. Both waiting time and runtime exhibit **heavy-tailed distributions**.
3. Most jobs request **small resources**, but extreme requests exist.
4. Non-complete jobs account for $\tilde{2}5\%$ of all jobs!
5. Individual users create spikes of failing jobs, which put a strain on cluster resources - better education on careful review before submitting jobs might be necessary.

- **Takeaway:** The system is **not homogeneous**, variability is fundamental.
- **Next:** We investigate the **sources of this heterogeneity** in the following section.

Outline

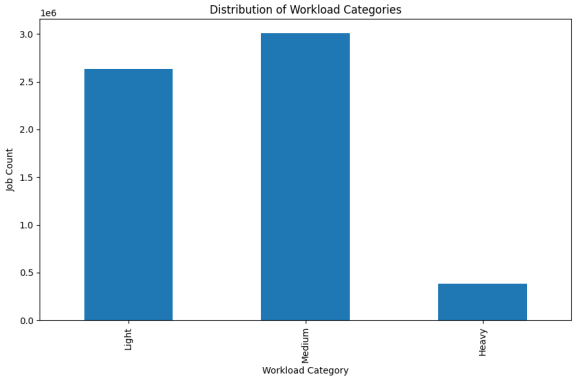
Workload Heterogeneity: CPU, Mem, GPU

Motivation: Workload is **multi-dimensional**, no single variable can adequately represent it.

Approach: Construct a workload variable via **clustering on resource request features**

- $\log(1 + x)$ transform + standardization
- K-means clustering, elbow method select $k = 3$

Workload Categories



Labels are assigned by ranking cluster centers by resource intensity.

1. **Light:** low CPU/memory, no GPU
2. **Medium:** CPU / memory intensive
3. **Heavy:** GPU-intensive, high memory

Observation:

- Most jobs are **light or medium**
- Heavy jobs are **less frequent**

Implication:

- A small number of **resource-intensive jobs** may drive system behavior

Heavy User Dominance

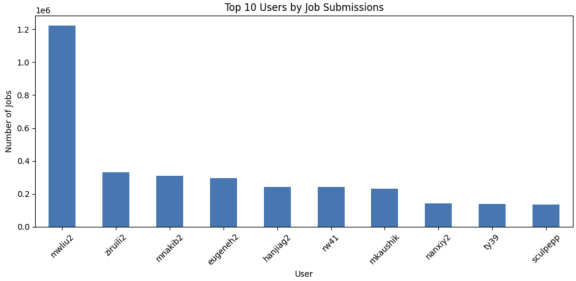


Figure 3: Top users dominate job submissions, showing heavy workload concentration

- Top 10% users contribute 93.66% of total job submissions
- Job distribution is highly skewed (long-tail behavior)
- Majority of users generate minimal workload

Insight:

- Cluster workload is dominated by a small group of heavy users

Impact:

- Increased scheduling pressure and reduced fairness

GPU Resource Concentration

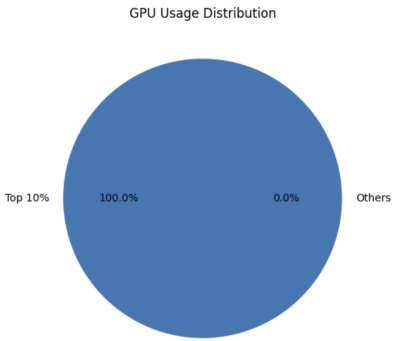


Figure 4: GPU usage is almost entirely dominated by a small group of users

- Top 10% users consume 100% of GPU resources
- GPU usage is significantly more concentrated than CPU usage
- Majority of users have little or no GPU access

Insight:

- Extreme resource concentration $\rightarrow$ near-monopoly

Impact:

- GPU queue congestion and longer waiting times

Department-Level Concentration

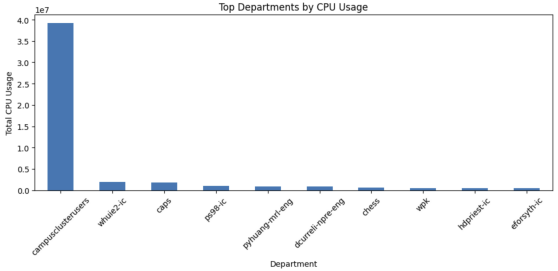


Figure 5: CPU usage is highly concentrated among a few departments

- Top 5 departments account for 82.98% of total CPU usage
- Resource demand is concentrated within few research groups
- Limited contribution from smaller departments

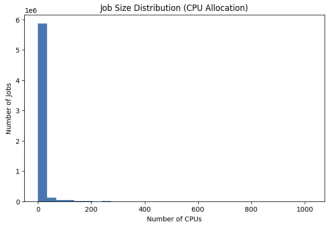
Insight:

- Institutional-level imbalance in resource utilization

Impact:

- Reduced accessibility for smaller or emerging departments

Job Size Distribution



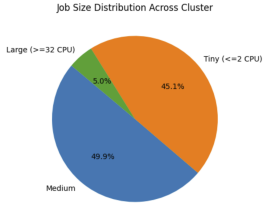


Figure: Job size distribution shows imbalance between tiny and large jobs

- 45.15% jobs are tiny (2 CPUs) → high scheduling overhead
- Only 4.99% jobs are large (32 CPUs) → high resource blocking
- Workload is highly polarized

Insight:

- Mixed job sizes create inefficiencies in scheduling

Impact:

- Increased queue delays and reduced system throughput

New vs Existing Users

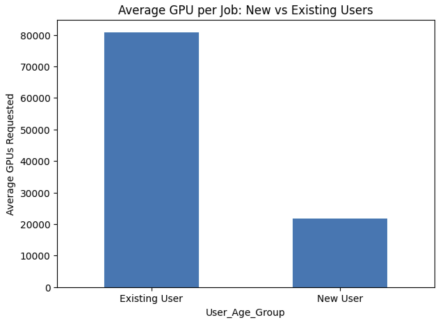


Figure: Existing users dominate GPU usage compared to new users

- **Existing users** consume significantly more GPU resources
- **New users** show lower GPU usage and engagement
- Usage gap indicates experience-based advantage

Insight:

- Entry barrier for new users in accessing GPU resources

Impact:

- Slower onboarding and reduced adoption of advanced computing

Inequality Metrics

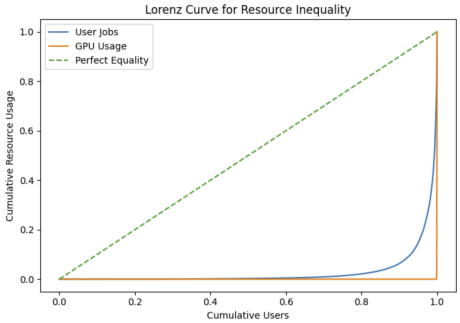


Figure: High Gini values confirm extreme inequality in resource distribution

- **User job inequality (Gini):** 0.945
- **GPU usage inequality (Gini):** 0.998
- Values close to 1 indicate extreme inequality

Insight:

- Resource distribution is highly uneven across users

Impact:

- System exhibits near-monopoly behavior in resource usage

Section Summary

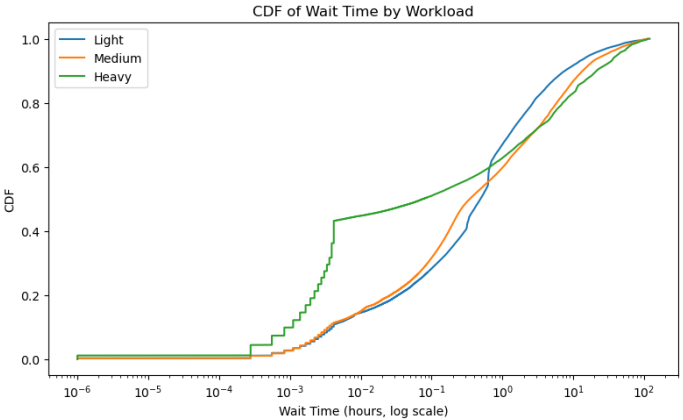
Key Findings:

1. **Top 10% users contribute → 93.66% jobs** Strong workload skew
2. **Top 10% users consume → 100% GPU** Near-monopoly behavior
3. **Top 5 departments account for → 82.98% CPU** Centralized demand
4. **45.15% tiny vs 4.99% large jobs** Overhead + resource blocking
5. **Existing users dominate GPU access** Entry barrier for new users
6. **Gini: User 0.945 — GPU 0.998** Severe inequality confirmed

- **Takeaway:** Heterogeneity arises from **workload structure** and **user behavior**
- **Next:** Leads to **resource contention** and **queue congestion**

Outline

Workload vs Waiting Time: Who Competes for Resources?



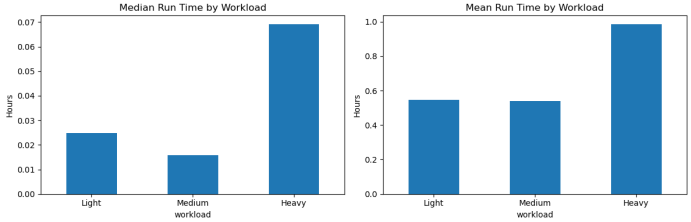
Key Observations:

- Most jobs start quickly
- A subset experiences **extreme delays**
- Heavy jobs have a strong **right-tail behavior**

Interpretation:

Contention is **concentrated in the tail, not the average**

Workload vs Runtime: Why Does It Happen?



Key Observations:

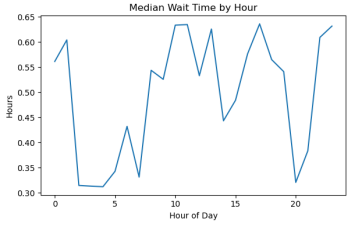
- Heavy jobs have **longer runtimes**
- CPU jobs are shorter and more homogeneous

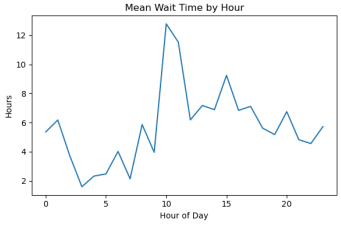
Interpretation:

- Long jobs **lock resources**
- Reduce system availability
- Intensify competition

Long-running heavy jobs drive resource contention

Submission Time vs Waiting Time: When Does the Queue Break Down?





Key Observations:

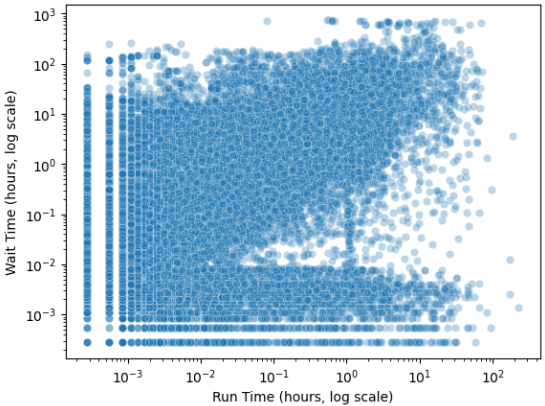
- Mean wait time shows clear spikes
- Extreme delays occur at specific hours

Interpretation:

- Most jobs are not strongly affected
- Congestion appears in the **tail**
- Peak periods create severe delays

Queue congestion is driven by extreme delay events, not typical jobs

Runtime vs Waiting Time: Why Is It Hard to Predict?



Key Observations:

- They are **positively** related
- Spearman correlation: 0.4238
- The scatter remains highly dispersed

Interpretation:

- Runtime alone cannot explain delays
- Congestion emerges from other factors (e.g. workload, users, and resource availability)

Section Summary

Key Findings:

1. Heavy jobs exhibit **extreme tail delays** - most jobs start quickly, but a subset faces severe wait times driven by long-running workloads locking resources.
2. Queue congestion is **temporally concentrated** - mean wait time spikes at specific hours, indicating burst-driven overload rather than uniform pressure.
3. Runtime and wait time are **positively but weakly correlated** (Spearman $= 0.4238$) - delays cannot be explained by job length alone; system context matters.

- **Takeaway:** Contention and congestion are **system-level consequences** of heterogeneity.
- **Next:** We examine how these effects lead to **system inefficiency**.

Outline

System Inefficiency - slurm Job Data

Key Findings

- Median utilization: 0.57%
- 95.5% of jobs use less than 50% of requested time

Interpretation

- Jobs are typically 200x shorter than requested
- Indicates systematic overestimation across users

Scale of Inefficiency:

- Median runtime: 1.2 minutes vs 240 minutes requested
- Median utilization: 0.57%
- Roughly 800 million CPU-hours reserved but unused

Overestimation Across Job Sizes:

- 5-minute jobs: 11.1x overestimation
- 10-minute jobs: 7.5x

Takeaway:

- Inefficiency is both **large-scale** and **systematic**

Is Inefficiency Limited to Runtime?

So far:

- Significant runtime overestimation across jobs
- Large-scale underutilization of reserved time

New Question:

- Is this inefficiency specific to runtime?
- Or does it extend to other resources?

Next Step:

- Analyze CPU and memory efficiency using seff data

System Inefficiency - seff data

Job-Level Efficiency Records:

- Each row corresponds to a completed job
- Includes both requested and actual resource usage

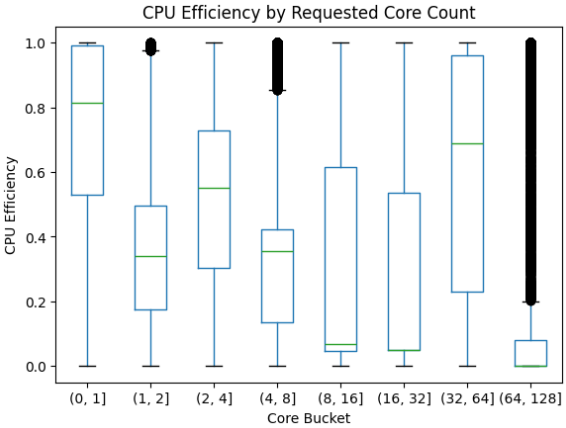
Key Fields:

- CPU utilization and efficiency
- Memory usage and efficiency

Why this matters:

- Captures how much of allocated resources were actually used
- Enables direct measurement of CPU and memory inefficiency

System Inefficiency - CPU efficiency in seff data



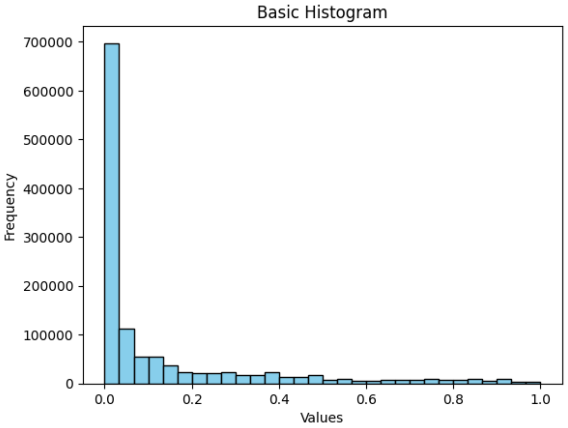
Key Observations:

- Single-core jobs show relatively high efficiency (median ~ 0.81)
- Mid-sized jobs (8–32 cores) have very low efficiency
- Efficiency varies significantly across core allocations

Implication:

- Users tend to over-request CPUs, especially for mid-sized workloads

System Inefficiency - CPU efficiency in seff data



Key Observations:

- Median memory efficiency: 1.48%
- Interquartile range: 0.02% – 15.02%
- 91.5% of jobs use less than 50% of requested memory
- Memory utilization is extremely low across most jobs

Implication:

- Memory over-allocation is widespread and consistent

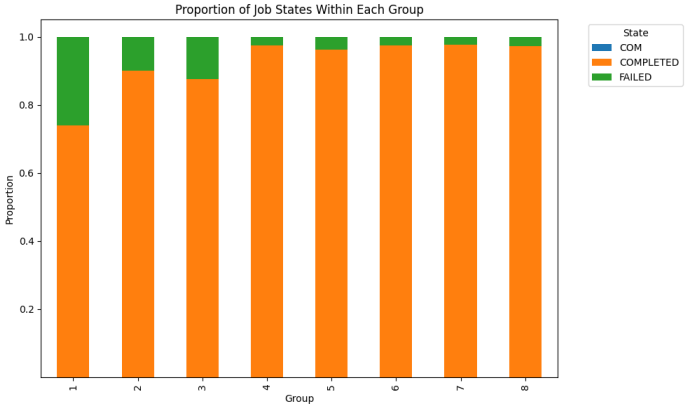
System Inefficiency - Job Categories: An Efficiency Metric

Motivation: Single efficiency metrics miss the full picture — a job can be CPU-efficient but wasteful of memory or time.

Group	Memory	CPU	Time	Interpretation
1	L	L	L	Fully inefficient
2	L	L	H	Only time-efficient
3	L	H	L	Only CPU-efficient
4	L	H	H	CPU + time efficient, memory wasteful
5	H	L	L	Only memory-efficient
6	H	L	H	Memory + time efficient, CPU wasteful
7	H	H	L	Memory + CPU efficient, time wasteful
8	H	H	H	Fully efficient

Groups capture heterogeneous efficiency profiles, not just overall performance

System Inefficiency - Job Category Distribution and Outcomes



Key Observations:

- Groups 3 and 7 dominate — high CPU efficiency is common
- Group 8 (fully efficient) is rare
- Failed jobs concentrate in **low-efficiency groups**

Interpretation:

- Most jobs are efficient in **at most one or two** dimensions
- Job failures are structurally linked to inefficiency

Efficiency profiles correlate with job outcomes

System Inefficiency - Job Category & Failure Rate: Chi-Squared Test

Group	Completed	Failed	Fail %
1 (L/L/L)	383,719	134,455	26.0%
2 (L/L/H)	9,268	1,017	9.9%
3 (L/H/L)	529,255	74,763	12.4%
4 (L/H/H)	12,600	318	2.5%
5 (H/L/L)	31,936	1,289	3.9%
6 (H/L/H)	5,942	152	2.5%
7 (H/H/L)	59,359	1,491	2.4%
8 (H/H/H)	5,450	151	2.7%

Overall Test:

$\chi^2 = 55{,}557.2$, df $= 7$, $p \approx 0$

Bonferroni-corrected comparison of 28 pairs:

Groups 4, 6, 7, 8 are **indistinguishable** ($p_{\text{Bonf}} = 1.0$)

Groups 1, 2, 3 differ significantly from all others

Setup:

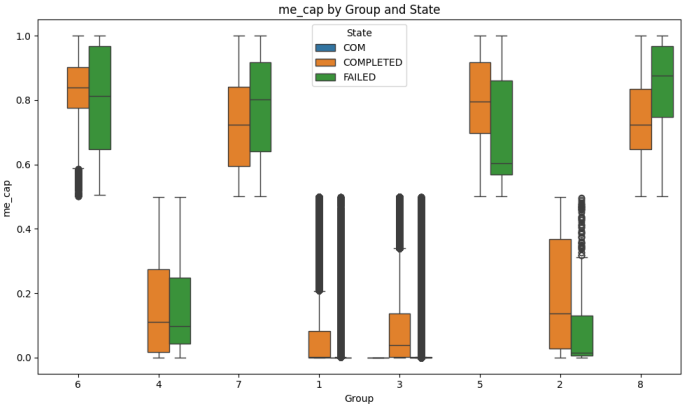
- Test whether failure rates differ across the 8 efficiency groups
- Pairwise χ^2 with Bonferroni correction

Key Findings:

- Groups 1, 2, 3 (low mem efficiency) have failure rates of 10–26%
- Groups 4, 6, 7, 8 (high mem efficiency) $\sim$2.5% and are statistically identical
- CPU and time efficiency do **not independently** predict failure

Memory efficiency is necessary to suppress job failures

System Inefficiency - Within-Group Efficiency Distributions



Key Observations:

- H-mem groups concentrate near $[0.6, 1.0]$
- L-mem groups collapse near zero

Interpretation:

- The 0.5 threshold creates **clean, meaningful separation**
- Failed jobs in efficient groups failed due to **other bottlenecks**, not memory waste

Group membership captures memory waste - failures are not random across groups

System Inefficiency - Improvements

Potential Approaches:

- Improve runtime and resource estimation
  - Historical data guidance
  - Educational resources
  - Lightweight estimation tools
- Encourage more accurate resource requests
- Introduce flexible runtime extensions
  - Reduce risk of job failure
  - Lower incentive to over-request

Potential Challenges:

- These solutions depend on user behavior and system changes
- Adoption and impact are not guaranteed

System Inefficiency - User Feedback

The Idea:

- Users are more engaged when given personalized insights
- Example: yearly recaps (e.g., Spotify Wrapped)
  - Users actively review, share, and discuss their data

Our Suggestion:

- Provide each user with a summary of their cluster usage:
  - Runtime efficiency
  - CPU efficiency
  - Memory efficiency
- Optional comparison:
  - Against overall cluster usage
  - Or similar user groups

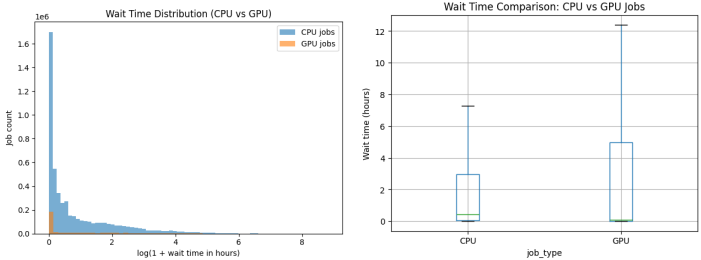
Implementation:

- Built on merged dataset (Slurm + Seff)
- Enables unified efficiency analysis per user

Outline

Waiting time prediction-Why Separate CPU and GPU Models?

- Waiting time distributions differ significantly (Mann–Whitney test, $p \approx 0$)
- GPU jobs have longer average waiting times
- Both distributions are highly skewed with long tails
- Different patterns suggest different underlying mechanisms



Conclusion: Build separate prediction models for CPU-only and GPU jobs.

Waiting time prediction-Some settings

Predictors we used:

- Categorical features: 'SubmitWeekday', 'SubmitHour', 'QOS';
- Numeric features: 'GPU_Count', 'ReqCPUS', 'ReqMem_MB', 'Priority';
- One-hot (categorical); z-score scaling (numeric)
- Data-driven feature selection: EDA + model-based evaluation.

Missing Values: **Numerical**: median imputation; **Categorical**: mode imputation.
Winsorization: cap extreme values using 2.5%–97.5% quantiles.
Evaluation metrics: MAE, RMSE, RMSLE.
Models were trained on the log-transformed response

$$\log(1 + \text{WaitTimeHr}).$$

After prediction, the fitted values were transformed back by

$$\widehat{\text{WaitTimeHr}} = \exp(\widehat{y}_{\log}) - 1.$$

GPU job waiting time prediction

Model used:

- Random Forest: 200 trees, max depth=10
- XGBoost: 300 trees, depth=5, learning rate=0.05

Prediction Results:

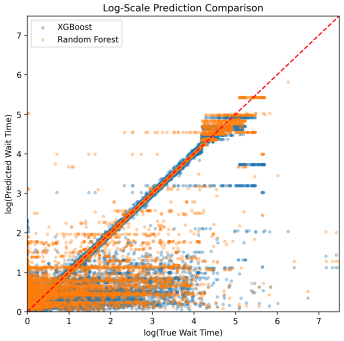
Model	MAE	RMSE	RMSLE
XGBoost	2.5583 hours	24.4489 hours	0.5377
Random Forest	1.9979 hours	21.9759 hours	0.5104

Table 1: Prediction performance of direct regression models without grouping.

- The Random Forest model performs slightly better than the XGBoost model.
- Random Forest has an RMSLE of 0.51, which means predictions are typically within a factor of about 1.67 of the true value.

GPU job waiting time prediction

- RF performs slightly better.
- Good fit in mid-range.
- High uncertainty for small values.
- Underestimation for large values.
- Likely due to skewness and high variability.



CPU-only job waiting time prediction

Definition of CPU-only jobs: jobs with `GPU_Count` $= 0$.
Model used: Random Forest: 200 trees, max depth=10.
Prediction Results:

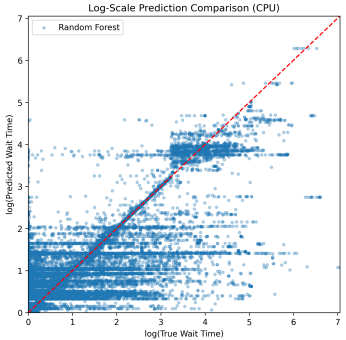
Metric	Value
MAE	3.9319 hours
RMSE	23.3144 hours
RMSLE	0.6559

Table 2: Prediction performance of the Random Forest model for CPU-only jobs.

- On average, predictions deviated by about 3.9 hours,
- Random Forest has an RMSLE of 0.66, meaning predictions are typically within a factor of about 1.9 of the true values.
- The CPU-only model has a larger MAE and RMSLE. CPU-only jobs' waiting times are harder to predict.

CPU-only job waiting time prediction

- Good fit in mid-range.
- High uncertainty for small values.
- Underestimation for large values.
- Likely due to high variability in CPU-only jobs' waiting times.



Outline

Summary & Conclusions

1. **The system is efficient on average, but driven by tail behavior**
   Most jobs are small and start quickly - extreme delays and resource waste are concentrated in a minority of jobs that disproportionately shape system load.
2. **Heterogeneity is structural, not incidental**
   Top 10% of users account for 93.66% of jobs and nearly all GPU usage (Gini $= 0.998$)
3. **Contention and congestion are system-level consequences of heavy jobs**
   Long-running GPU-intensive jobs lock resources and create extreme spikes in waiting time. Runtime alone (Spearman $= 0.42$) cannot explain delays - further information on the system is needed to model this.

Summary & Conclusions (cont.)

1. **Resource waste is widespread and multidimensional**
   Median memory efficiency is 1.48%; ∼800 million CPU-hours reserved but unused. Inefficiency is not limited to one resource - our 8-group job category metric of efficiency reveals that most jobs are efficient in at most one or two dimensions.
2. **Job failures are not random - they are structurally linked to efficiency profiles**
   Failed jobs concentrate in low-efficiency groups. Jobs that fail within high-memory groups do so due to other bottlenecks, confirming that the efficiency taxonomy captures meaningful behavior.
3. **Waiting time is predictable but inherently noisy**
   Random Forest achieves RMSLE of 0.51 (GPU) and 0.66 (CPU-only). Mid-range predictions are reliable; tail behavior remains hard to capture due to high variability.

Recommendations

1. **Resource-Aware Scheduling Guidance**
   Provide job-submission tooling that uses historical usage data to suggest realistic CPU, memory, and walltime requests
2. **Mitigate Resource Concentration**
   Introduce fairshare or quota policies targeting the top users and departments to broaden access, particularly for GPU resources and new users.
3. **Leverage Predictive Models for Scheduling**
   Integrate trained wait-time models into the scheduler to flag delay-prone submissions and reorder or backfill queues more effectively during peak hours.
4. **Personalized Efficiency Feedback**
   Report per-user efficiency summaries (runtime, CPU, memory) compared against the overall cluster, increasing awareness and incentivizing more accurate resource requests.
5. **Targeted Intervention by Efficiency Group**
   Use the 8-group job category efficiency metric to identify systematically wasteful job profiles and deliver group-specific guidance, prioritizing users whose jobs consistently fall in Groups 1–3.

Thank you!

We would like to sincerely thank the **Illinois Campus Cluster team** for providing the data and the opportunity to work on this project, and for allowing us to gain valuable insights into real-world problems.

Questions?